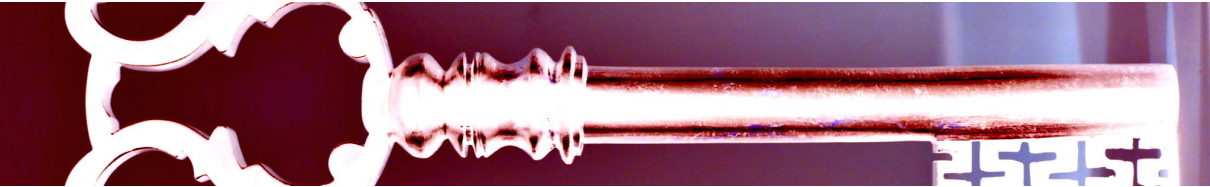
universität innsbruck



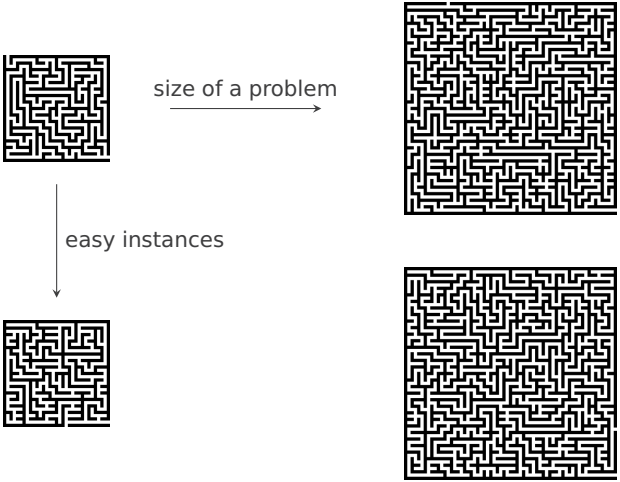
Computational Complexity

Cryptography

Rainer Böhme

30 April 2026

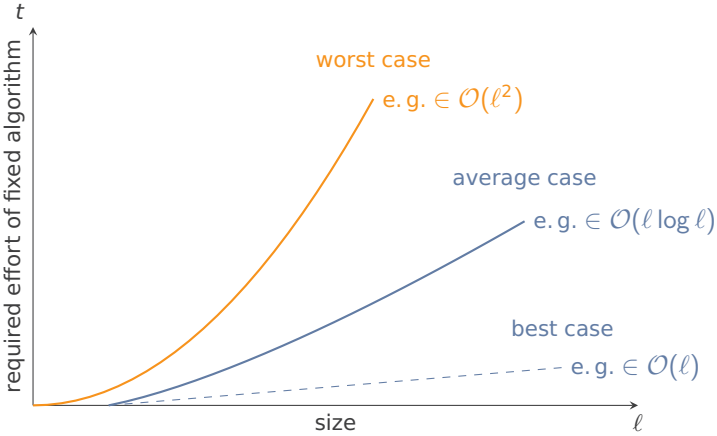
How Hard Is a Problem ?



Agenda

1. **Models of computation** $\sim$ adversarial capability
2. Complexity classes $\sim$ defender margin
3. Selected problems with applications to cryptography

Effort to Solve a Problem



Landau “Big-O” Notation

Consider two functions $f(\ell)$ and $g(\ell)$:

$$f(\ell) \in \mathcal{O}(g(\ell)) \quad \Leftrightarrow \quad \limsup_{\ell \to \infty} \left| \frac{f(\ell)}{g(\ell)} \right| < \infty$$

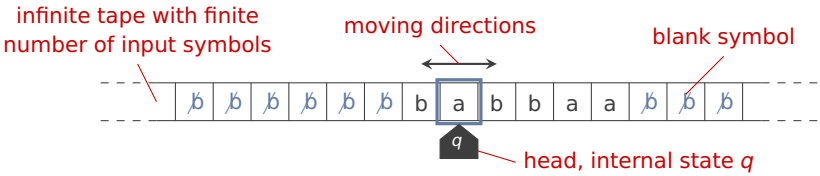
“$g(\ell)$ is an **asymptotic upper bound** of $f(\ell)$ as $\ell \to \infty$.”

Properties

- Upper bound: $f(\ell) \leq C \cdot g(\ell)$
- Asymptotic: only for “sufficiently large” $\ell > \ell_0 \in \mathbb{R}^+$
- We are interested in the shape. Hence we allow for a positive scaling constant C.

Turing Machine

The Turing machine is an abstract model of computation.



A Turing machine is defined by the tuple $\mathcal{A} = (\mathcal{Q}, \Sigma, \Gamma, q_0, \mathcal{F}, \Delta)$:

- $\mathcal{Q}$: finite number of states
- Σ: input alphabet
- $\Gamma = \Sigma \cup \{\not{b}\}$: tape alphabet
- $q_0 \in \mathcal{Q}$: initial state
- $\mathcal{F} \subseteq \mathcal{Q}$: set of final states
- $\Delta \subseteq \mathcal{Q}^2 \times \Gamma^2 \times \{\leftarrow, \rightarrow, \perp\}$: transitions

Turing 1936

State Transition

$$(q, q', a, a', \{\leftarrow, \rightarrow, \perp\}) \in \Delta$$

If Turing machine $\mathcal{A}$ is in state q and the head points to symbol a, it replaces a by a', changes to state q', and then moves the head one step to the left ($\leftarrow$), right ($\rightarrow$), or not at all ($\perp$).

We distinguish:

- deterministic Turing machine (DTM): for every tuple $(q, a) \in \mathcal{Q} \times \Gamma$ there exists at most one tuple $(q, q', a, \dots) \in \Delta$.
- nondeterministic Turning machine (NTM): every possible path is explored simultaneously
  $\longrightarrow$ NTMs are purely theoretical. No practical computer parallelizes this way.

Turing completeness

A set of data manipulation rules is called Turing-complete if and only if it can simulate any DTM.

Important Notions

computability The machine reaches a final state $q \in \mathcal{F}$ at all.

$\updownarrow$

complexity Measure the effort needed to reach the final state.

- specific algorithm
- hardness of the problem
  best algorithm in worst case
- time
  DTM: steps until final state
- space
  DTM: tape cells used

Agenda

1. Models of computation ∼ adversarial capability
2. **Complexity classes** ∼ defender margin
3. Selected problems with applications to cryptography

Boolean Satisfiability (SAT)

Decision problem

Given a Boolean expression with n variables x_i, is it possible to find an assignment of the variables that evaluates to 1 (true)?

Examples

1. $x_1 \cdot \overline{x_2}$ $\rightarrow$ satisfiable ($x_1 = 1$ and $x_2 = 0$)
2. $x_1 \cdot \overline{x_1}$ $\rightarrow$ not satisfiable (complements axiom)
3. $(x_1 \cdot x_2) + \overline{x_3}$ $\rightarrow$ satisfiable (e. g., $x_1 = 1$, $x_2 = 1$ and $x_3 = 0$)

Real problems have $n = 100$ and more variables.

Best known algorithm

Check the truth table. In the worst case, we have to go through all 2^n rows: $\rightarrow O(2^n)$.

How can we know that there are no better algorithms?

Note: Here we use Boolean logic, not arithmetic in GF(2).

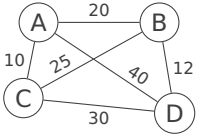
Traveling Salesperson (TSP)

Decision problem

Given a fully connected weighted graph with n nodes, and a number k, is there a cycle that passes through each node exactly once and has a total weight of at most k?

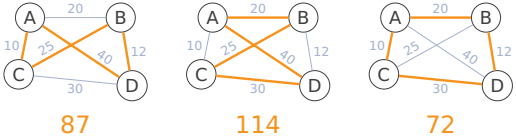
Example

$(n = 4)$



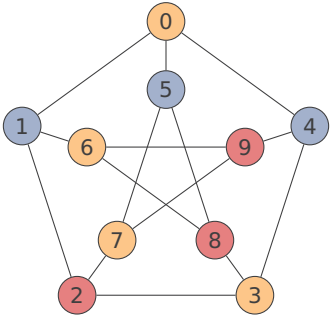
Best known algorithm

We have to check all $\frac{(n-1)!}{2}$ possible tours: $\rightarrow O(n!)$.



Graph 3-Coloring (G3C)

Graph G. (Example: Petersen graph)



Coloring Assign a color to each node such that adjacent nodes differ.

3-coloring $G \in \text{G3C} \Leftrightarrow \exists$ coloring with 3 colors

Hardness of Problems

may not be known !

Definition
The hardness of a problem is the worst-case time complexity of the **best** algorithm.

SAT, TSP, and G3C have common properties:

1. They are decision problems. This means we can map them to formal languages: We write instances as sequences $\in \Sigma^*$. "Yes" decisions are words of a language $\mathcal{L}$.

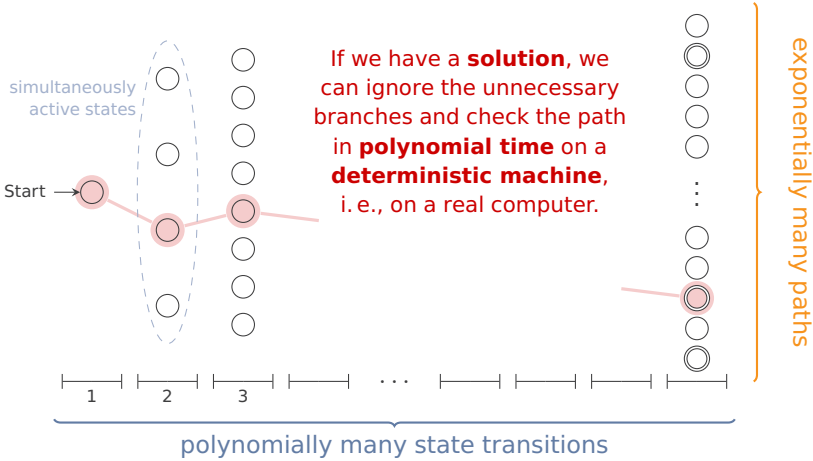
   $\rightarrow$ We choose deterministic Turing machines (DTM) as computational model because they represent what every computer can do.

2. Efficient algorithms are known to **verify** a given solution.

   $\rightarrow$ Purely theoretical nondeterministic Turing machines (NTM) can **decide** such problems in polynomial time.

Intuition

Nondeterministic machines can transition into multiple states “in parallel.”



Complexity Classes

Complexity classes are sets of problems.

Definitions

$$DTIME(f(n)) := \{L \in \Sigma^* \mid \exists\, f(n)\text{-time-bounded DTM which decides } L\}$$

$$NTIME(f(n)) := \{L \in \Sigma^* \mid \exists\, f(n)\text{-time-bounded NTM which accepts } L\}$$

$$DSPACE(f(n)) := \{L \in \Sigma^* \mid \exists\, f(n)\text{-space-bounded DTM which decides } L\}$$

$$NSPACE(f(n)) := \{L \in \Sigma^* \mid \exists\, f(n)\text{-space-bounded NTM which accepts } L\}$$

Some obvious relations

$$DTIME(f(n)) \subseteq DSPACE(f(n)) \subseteq NSPACE(f(n))$$

$$DTIME(f(n)) \subseteq NTIME(f(n))$$

Polynomial Bounds

Definitions

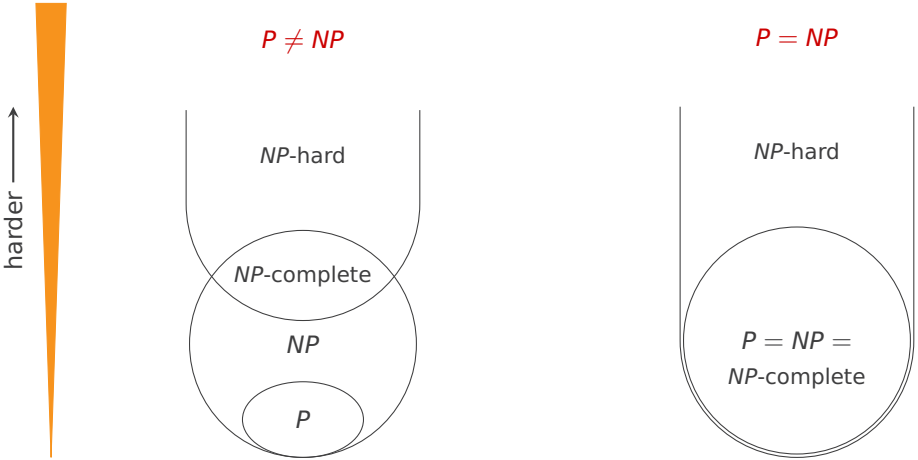
$$P := \bigcup_{p \text{ polynomial in } n} DTIME(p(n))$$

$$NP := \bigcup_{p \text{ polynomial in } n} NTIME(p(n))$$

Why polynomial?

- All computation models of real computers can be simulated by a deterministic Turing machine with at most polynomial overhead.
- Closure: insert any polynomial into any polynomial, you get a polynomial.
- Polynomials grow gently in n. Therefore problems in P are called tractable.
- NP contains many problems for which the best known DTM-algorithm needs exponential time. These (and harder) problems are called intractable.

P–NP Problem


harder
$P \neq NP$
NP-hard
NP-complete
NP
P
$P = NP$
NP-hard
$P = NP =$
NP-complete

Million Dollar Question

Quick career path

- If you find an algorithm that solves every instance of any NP-complete problem in polynomial time on a classical computer, you have proven that $P = NP$.
- Since no such algorithm has been found since the 1960s, most experts conjecture that $P \neq NP$.
- But that hasn't been proven either. If you can find that proof, you also get at least a million dollars.

How do we know that a problem is in NP?

1. Show that it's not harder by finding a polynominal algorithm to verify the solution.
2. Show that it's not weaker by analysing the best algorithm . . .

Reduction

How to determine the hardness of a new problem L^* ?

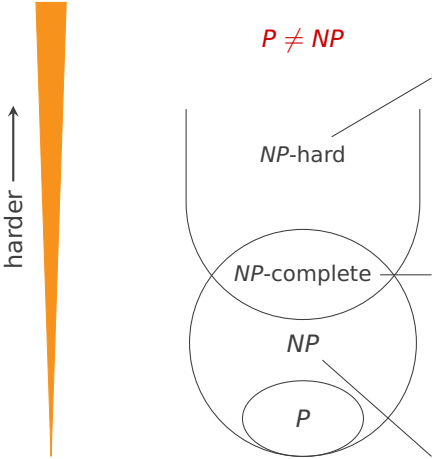
Idea: If we find an easy transformation that reformulates every instance of a hard problem L as a solvable instance of L^*, then L^* must be at least as hard as the known problem L.

$$L \preceq_{\mathrm{m}} L^*$$

Type of reduction
qualifies "easy" transformations:
p – polynomial (Cook–Turing)
m – mapping (Karp)

"L is reducible to L^*." $\Leftrightarrow$ "L cannot be harder than L^*."

P–NP Problem



at least as hard as any other problem in *NP*

Proof that L^* is *NP*-complete (by reduction)

Require: *NP*-complete problem L of size n
  . . . {do something}
  for at most $\{\text{poly}(n) \mid 1\}$ times **do**
    . . . {do something}
    call solution oracle for L^*
    . . . {do something}
  end for
  . . . {do something}
  return decision on L

"**decide** in more than polynomial time, **verify** in polynomial time"

Agenda

1. Models of computation $\sim$ adversarial capability
2. Complexity classes $\sim$ defender margin
3. **Selected problems with applications to cryptography**

Integer Operations

Given $a, b \in \mathbb{Z}$, it is possible to compute the following functions in polynomial time of the size (e. g., in bits) of their arguments:

- Sums $a + b$ and differences $a - b$;
- Products $a \cdot b$;
- Division with remainder $a = q \cdot b + r$ (with $r < b$ and all integers positive);
- Greatest common divisors $\gcd(a, b)$;
- Finding integers u, v such that $u \cdot a + v \cdot b = \gcd(a, b)$.

Modular Operations

Given $a, b, n \in \mathbb{Z}$, $n > 1$, it is possible to perform the following operations in polynomial time of the size of their arguments:

- Computing modular reductions $a \mod n$;
- Computing sums $(a + b) \mod n$, differences $(a - b) \mod n$, and the products $a \cdot b \mod n$;
- Determining whether a is invertible modulo n;
- Computing the multiplicative inverse $a^{-1} \mod n$ if it exists;
- Computing the exponentiation $a^b \mod n$.

cf. Katz & Lindell 2014, p. 548

Selected Number-Theoretic Problems

Factorization	
Problem	Given semiprime n, find p and q such that $n = p \cdot q$, $\ell = \lceil \log_2 n \rceil$, $\log_2 p \approx \log_2 q$, $p \neq q$
Best known algorithm	general number field sieve
Complexity	$e^{\mathcal{O}\left(\ell^{\frac{1}{3}} \cdot (\log \ell)^{\frac{2}{3}}\right)}$

Discrete Logarithm in $\mathbb{Z}_p$	
Problem	Given element a of a multiplicative group of integers modulo prime p with generator g, find k such that $a \equiv g^k \mod p$
Known algorithms	baby step-giant step, <u>index calculus algorithm</u>
Complexity	$2^{\mathcal{O}\left(\ell^{\frac{1}{2}} \cdot (\log \ell)^{\frac{1}{2}}\right)}$ where $\ell = \lceil \log_2 p \rceil$

Further Reading

Textbooks:

- C. Papadimitriou (1994): Computational Complexity, Addison Wesley, 523 pages.
- S. Arora & B. Barak (2009): Computational Complexity: A Modern Approach, Cambridge University Press, 594 pages.

Brainteaser for the summer vacation:

- S. Aaronson (2011): Why Philosophers Should Care About Computational Complexity.

Syllabus

Summer Term 2026

	05.03.26	1.	Introduction, basic terminology	
	12.03.26	2.	Probability, information, and coding theory for cryptography	
	19.03.26	3.	Symmetric encryption, perfect secrecy	
Tue	24.03.26	4.	Block ciphers	HSB 9
	16.04.26	5.	Cryptanalysis of classical ciphers	
	23.04.26	6.	Number theory for cryptography	
	30.04.26	7.	Computational complexity for cryptography	
	07.05.26	8.	Asymmetric encryption	
Tue	19.05.26	9.	Message authentication	HSB 9
	28.05.26	10.	Elliptic curve cryptography	
	11.06.26	11.	Commitments, zero-knowledge, and secret sharing	
Tue	23.06.26	12.	Q & A for the exam	HSB 9
	25.06.26		**Written exam**	

Last update: April 30, 2026